Report HW 8 ECE-111

Name: Jasper Huang

PID: A17796149

Handshake Synchronizer

SystemVerilog Design Code (Student added parts)

handshake_synchronizer.sv

```
// input data register (register input data_in to data_in_reg)
always_ff@(posedge src_clk, posedge src_reset) begin
  // Student to add code here
  if (src_reset) begin
    data_in_reg <= 0;
  end else if (start && ready) begin
    data_in_reg <= data_in;
  end
end

// Data Latch :
// Control sending data from input data register through data latch
assign data = (data_out_en==1) ? data_in_reg : data;

// output data register (destination register to load data sent by sender_fsm)
always_ff@(posedge dest_clk, posedge dest_reset) begin
  if (dest_reset) begin
    data_out <= 0;
  end
  else if (data_in_en) begin
    data_out <= data;
  end
end
```

```
// Sender handshake synchronizer FSM instance
// Note : connect req_o to req_o and ack_i to sync_ack_i
sender_fsm sender_fsm_inst(
  // Student to add code here
  .src_clk(src_clk), .src_reset(src_reset),
  .start(start),
  .ready(ready),
  .data_out_en(data_out_en),
  .req_o(req_o),
  .ack_i(sync_ack_i)
);

// Receiver handshake synchronizer FSM instance
// Note : connect req_i to sync_req_i and ack_o to ack_o
receiver_fsm receiver_fsm_inst(
  // Student to add code here
  .dest_clk(dest_clk), .dest_reset(dest_reset),
  .req_i(sync_req_i),
  .ack_o(ack_o),
  .data_in_en(data_in_en)
);
```

```
// 2ff synchronizer instance for ack_o coming from receiver_fsm
// output of this synchronizer sync_ack_i will be connected to sender_fsm instance ack port
flipflop_synchronizer #(.WIDTH(1), .NUM_OF_STAGES(2)) ack_2ff_sync_inst
(
  // Student to add code here
  .clock(src_clk), .reset(src_reset),
  .d(ack_o),
  .q(sync_ack_i)
);

// 2ff synchronizer instance for req_o coming from sender_fsm
// output of this synchronizer sync_req_i will be connected to receiver_fsm instance ack_i port
flipflop_synchronizer #(.WIDTH(1), .NUM_OF_STAGES(2)) req_2ff_sync_inst
(
  // Student to add code here
  .clock(dest_clk), .reset(dest_reset),
  .d(req_o),
  .q(sync_req_i)
);
```

sender_fsm.sv

```
IDLE: begin
  // Student to add code here
  req_o <= 0;
  ready <= 1;
  data_out_en <= 0;
  if (start) begin
    state <= REQ_ACK_PHASE1;
  end else begin
    state <= IDLE;
  end
end
// Generate req_o = 1 to indicate handshake_rx_fsm that data is available
// Generate data_out_en = 1 to data_out_en data buffer and send data to the receiver logic
// Generate ready = 0 indicating no new req_ouest data will be accepted by tx_fm until current data is
// received by the receiver rx_fsm
// Wait for assertion of ack_i from handshake_rx_fsm
REQ_ACK_PHASE1: begin
  // Student to add code here
  req_o <= 1;
  ready <= 0;
  data_out_en <= 1;
  if (ack_i) begin
    state <= REQ_ACK_PHASE2;
  end else begin
    state <= REQ_ACK_PHASE1;
  end
end
// De-assert buffer data_out_en to 0 to latch current data until it is received by the receiver
// Continue to de-asserting of ready to 0 indicating no new data can be accepted by sender_fsm
// Wait for de-assertion of ack_i from handshake_rx_fsm
REQ_ACK_PHASE2: begin
  // Student to add code here
  req_o <= 0;
  ready <= 0;
  data_out_en <= 0;
  if (!ack_i) begin
    state <= IDLE;
  end else begin
    state <= REQ_ACK_PHASE2;
  end
end
// In Default state move to IDLE state
default: begin
    state <= IDLE;
end
```

receiver_fsm.sv

```
// Wait for req_iuest data coming from tx_fsm until then do not
// enable loading of data in the data register
ACK_REQ_PHASE1: begin
   // Student to add code here
   ack_o <= 0;
   data_in_en <= 0;
   state <= ACK_REQ_PHASE1;
   if (req_i) begin
      state <= ACK_REQ_PHASE2;
   end
   else begin
      state <= ACK_REQ_PHASE1;
   end
end
// In response to req_i=1 from tx_fsm, generate ack_o = 1 and enable
// loading of data in destination register and then wait for req_i = 0
ACK_REQ_PHASE2: begin
   // Student to add code here
   ack_o      <= 1;
   data_in_en <= 1;
   if (!req_i) begin
      state <= ACK_REQ_PHASE1;
   end
   else begin
      state <= ACK_REQ_PHASE2;
   end
end
// In Default state move to IDLE state
default: begin
   state <= ACK_REQ_PHASE1;
end
```

flipflop_synchronizer.sv

```
module flipflop_synchronizer
#(parameter integer WIDTH=1, parameter integer NUM_OF_STAGES=2)
(
  input logic clock, reset,
  input logic [WIDTH-1:0] d,
  output logic [WIDTH-1:0] q
);

 logic[WIDTH-1:0] r[NUM_OF_STAGES-1:0];
 always_ff@(posedge clock, posedge reset) begin
  if(reset == 1) begin
     for(int i=0; i<NUM_OF_STAGES; i=i+1) begin
      r[i] <= 0;
    end
  end
  else begin
    r[0] <= d;
    for(int i=0; i<(NUM_OF_STAGES-1); i=i+1) begin
      r[i+1] <= r[i];
    end
  end
 end
 assign q = (reset == 0) ? r[NUM_OF_STAGES-1] : 0;
endmodule: flipflop_synchronizer
```

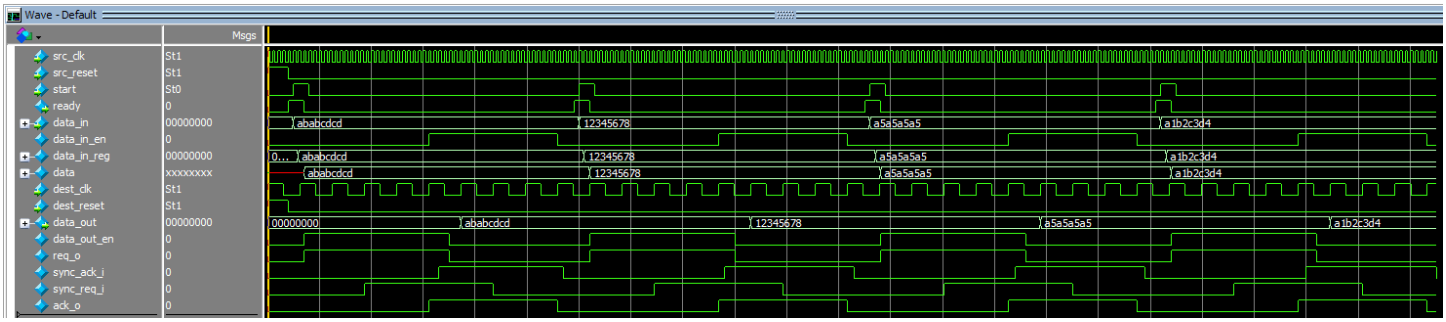
Synthesis resource usage snapshot

Analysis & Synthesis Resource Usage Summary

<<Filter>>

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	38
2		
3	Combinational ALUT usage for logic	37
1	-- 7 input functions	0
2	-- 6 input functions	0
3	-- 5 input functions	2
4	-- 4 input functions	0
5	-- <=3 input functions	35
4		
5	Dedicated logic registers	75
6		
7	I/O pins	70
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	src_reset~input
12	Maximum fan-out	42
13	Total fan-out	504
14	Average fan-out	2.00

Simulation snapshot



The simulation result shows the sending and receiving of four different data, where you can see for each data_in, the cycle and their corresponding flag is output correctly with the final data_out matching exactly as the data_in.